

Group members:

Ashwin Varkey 23115420

Chukwudi Williams 23061170

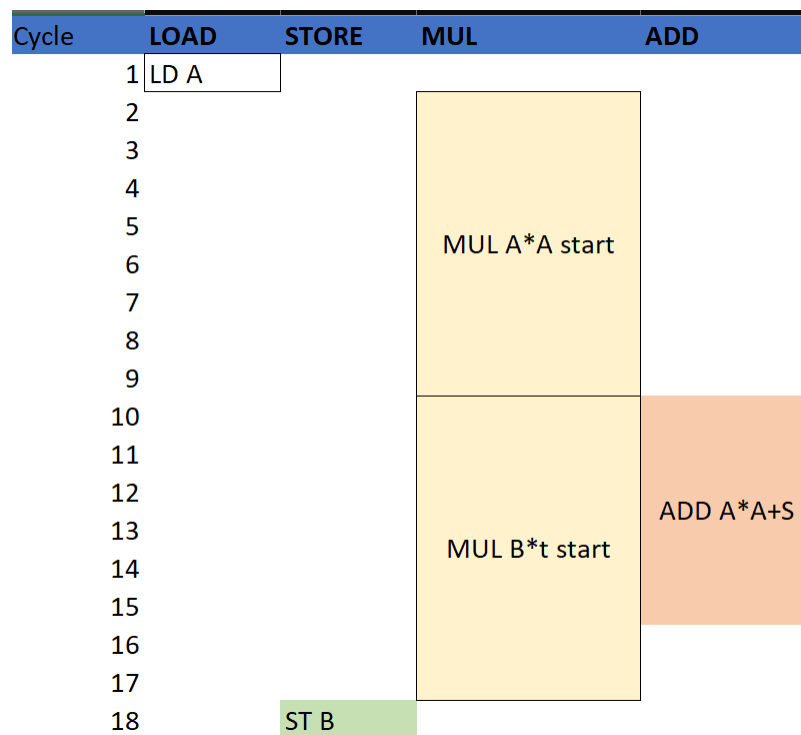
Manik Malik 23169717

Assignment 1: Code execution, proper microbenchmarking

1. (a) Assuming the compiler does not know about Modulo Variable Expansion (MVE) but otherwise produces perfect code, what is the hardware bottleneck that limits the performance of the code? Calculate the expected performance in flops/cy.

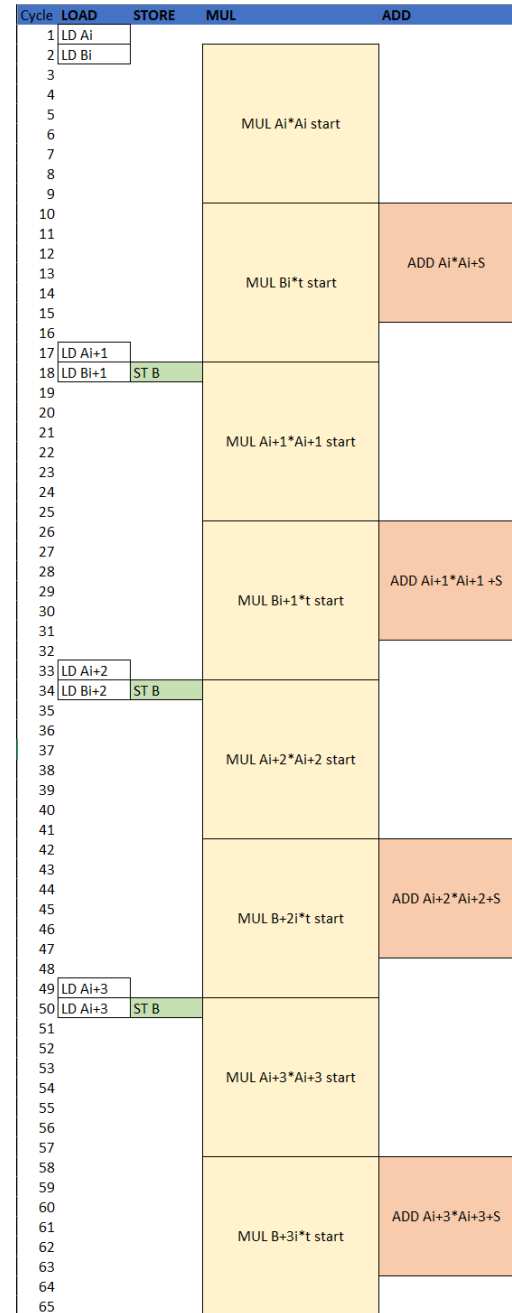
-We have 2 multiplication and 1 addition operation. The core can perform 1 MUL and 1 ADD together

The hardware bottleneck is the multiply instruction that uses 8 cycles per instruction and its dependency on the add instruction which cannot happen without the multiply first. So the total cycles as per the diagram below are 18 and 3 floating point operations are performed in these 18 cycles or 1 loop iteration = $3/18 = 1/6$ FLOPs



1. (b) Considering MVE

We have a limit of 4 instructions / cycle and a total of 16 floating point registers so the unrolling can be done. The unrolling of as much as 4 can be done but the point is that unrolling would not benefit us in the case of performance. As the add instruction is dependent on the multiply as shown in the figure below we cannot create a situation where ADD instruction is always in the pipeline as it is dependent on the multiply so add has to wait for the multiply. As shown in the image below the performance would still be let's say 65 cycle for 12 floating point operations = 0.1846 FLOPs



2. (a) Vector triad: $a[i] = b[i] + c[i] * d[i]$

- As described in the question, We initiated N with different values as follows using Haskell's list comprehension.

```
[round (1.5**k) | k <- [8,9..44]]
```

Which resulted in the following list

```
[26,38,58,86,130,195,292,438,657,985,1478,2217,3325,4988,7482,11223,16834,25251,37877,56815,85223,127834,191751,287627,431440,647160,970740,1456110,2184164,3276247,4914370,7371555,11057332,16585998,24878998,37318497,55977745]
```

We used the `scan_c.c` file provided in `~ptfs100h/GettingStarted/scan_c.c`. and modified as follows to be able to compute $a[i] = b[i] + c[i] * d[i]$. This can be seen in `ex_1_q1.c` file and is shown below.

```
icx -xHost -fno-alias ex_1_q21.c timing.c -o ex_1_q1.out
```

```

#include <stdio.h>
#include <stdlib.h>
#include "timing.h"

int main(int argc, char** argv) {

    double wct_start, wct_end;
    int id, nt, k, i;
    int NITER, N;
    double t = 0;
    const double s = 1.00000000001;

    printf("\n\nVector triad: a[i] = b[i] + c[i] * d[i]\n");
    printf("MFlop/s\t\t N\n\n");

    int rangeN[37] = {
        26,38,58,86,130,195,292,438,657,985,1478,2217,3325,4988,7482,11223,16834,25251,37877,
        56815,85223,127834,191751,287627,431440,647160,970740,1456110,2184164,3276247,4914370,
        7371555,11057332,16585998,24878998,37318497,55977745};

    for (int counter = 0; counter < 37; ++counter) {

        N=rangeN[counter];
        double *a = (double*)malloc(N * sizeof(double));
        double *b = (double*)malloc(N * sizeof(double));
        double *c = (double*)malloc(N * sizeof(double));
        double *d = (double*)malloc(N * sizeof(double));

        for (k = 0; k < N; ++k) {
            a[k] = 0.;
            b[k] = k + 1.;
            c[k] = k + 2.;
            d[k] = k + 3.;
        }

        NITER = 1;
        do {
            // time measurement
            wct_start = getTimeStamp();

            // repeat measurement often enough
            for (k = 0; k < NITER; ++k) {
                // This is the benchmark loop
                for (i = 0; i < N; ++i) {
                    // vector triad: a[i] = b[i] + c[i] * d[i]
                    a[i] = b[i] + c[i] * d[i];
                }
                // end of benchmark loop
                if (a[N/2] < 0.) printf("%lf", a[N/2]);
            }
            wct_end = getTimeStamp();
            NITER = NITER * 2;
        } while (wct_end - wct_start < 0.1); // at least 100 ms

        NITER = NITER / 2;

        double time = (wct_end - wct_start) / NITER;
        double MFlop = (2.0 * N * NITER) / (time * 1.0e6);
        double MFlopPerSec = MFlop / (1.0e-3 * NITER);

        printf("%f\t %d\n", MFlopPerSec, N);

        free(a);
        free(b);
        free(c);
        free(d);
    }
}

```

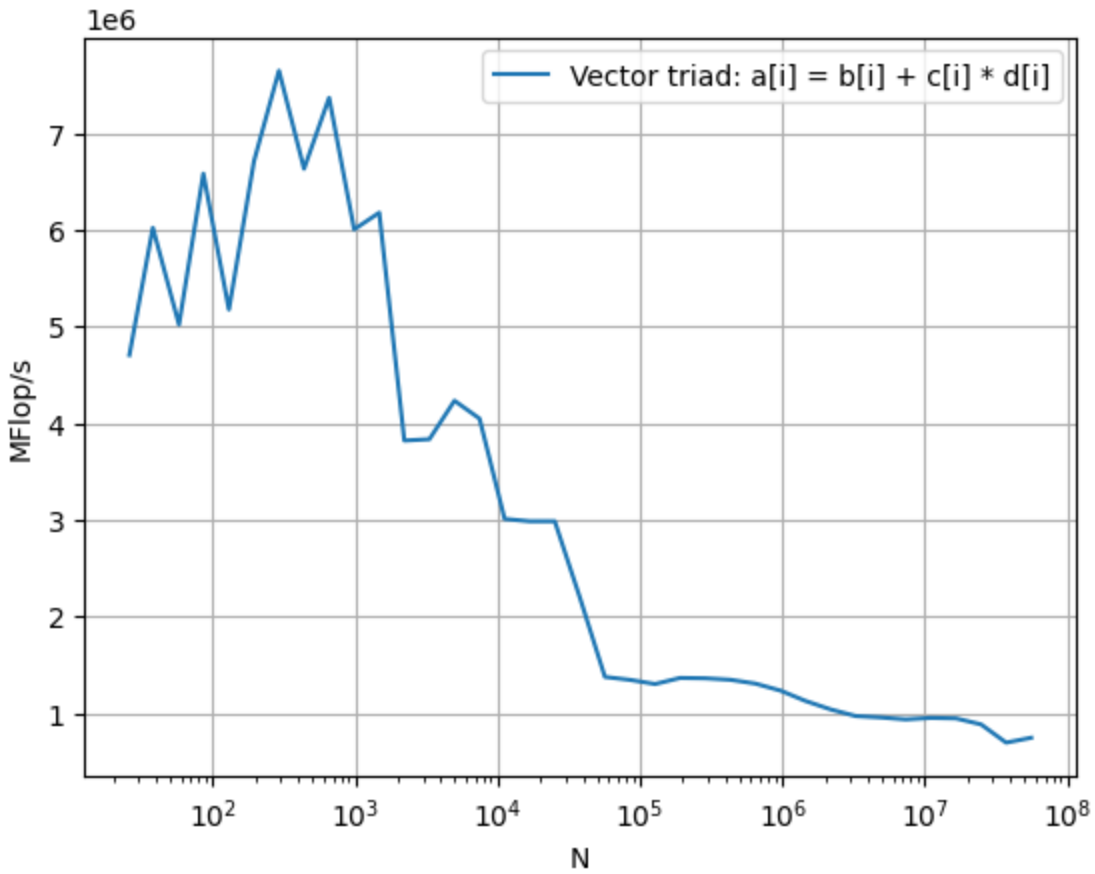
The result of the computation is a stream of table of data as shown below:

<u>MFlop/s</u>	<u>N</u>
4705141.844753	26
6023142.190182	38
5019747.939243	58
6583585.243796	86
5175821.097761	130
6712089.528479	195
7649194.037728	292
6633192.679428	438
7368528.507233	657
.	
.	
.	

The full list of the output is contained in `./data/v_triad.txt`

To be able to show the graph of the memory bandwidth with respect to the different N values of dynamic memory allocation, we employed Matplotlib from python.

The script we used is contained in `vector_triad_update.ipynb`.
Our obtained graph is shown below:



2(b) Vector update: $a[i] = s * a[i]$

We modified and repeated the computation to reflect vector update $a[i] = s * a[i]$

```
icx -xHost -fno-alias ex_1_q2.c timing.c -o ex_1_q2.out
```

```

#include <stdio.h>
#include <stdlib.h>
#include "timing.h"

int main(int argc, char** argv) {

    double wct_start, wct_end;
    int id, nt, k, i;
    int NITER, N;
    double t = 0;
    const double s = 1.00000000001;

    printf("\n\nVector update: a[i] = s * a[i] \n");
    printf("MFlop/s\t\t N\n\n");

    int rangeN[37] = {
        26,38,58,86,130,195,292,438,657,985,1478,2217,3325,4988,7482,11223,16834,25251,37877,
        56815,85223,127834,191751,287627,431440,647160,970740,1456110,2184164,3276247,4914370,
        7371555,11057332,16585998,24878998,37318497,55977745};

    for (int counter = 0; counter < 37; ++counter) {

        N=rangeN[counter];
        double *a = (double*)malloc(N * sizeof(double));

        for (k = 0; k < N; ++k) {
            a[k] = 0.;
        }

        NITER = 1;
        do {
            // time measurement
            wct_start = getTimeStamp();

            // repeat measurement often enough
            for (k = 0; k < NITER; ++k) {
                // This is the benchmark loop
                for (i = 0; i < N; ++i) {
                    // (b) vector update: a[i] = s * a[i]
                    a[i] = s * a[i];
                }
                // end of benchmark loop
                if (a[N/2] < 0.) printf("%lf", a[N/2]);
            }
            wct_end = getTimeStamp();
            NITER = NITER * 2;
        } while (wct_end - wct_start < 0.1); // at least 100 ms

        NITER = NITER / 2;

        double time = (wct_end - wct_start) / NITER;
        double MFlop = (N * NITER) / (time * 1.0e6);
        double MFlopPerSec = MFlop / (1.0e-3 * NITER);

        printf("%f\t %d\n", MFlopPerSec, N);

        free(a);
    }

    return 0;
}

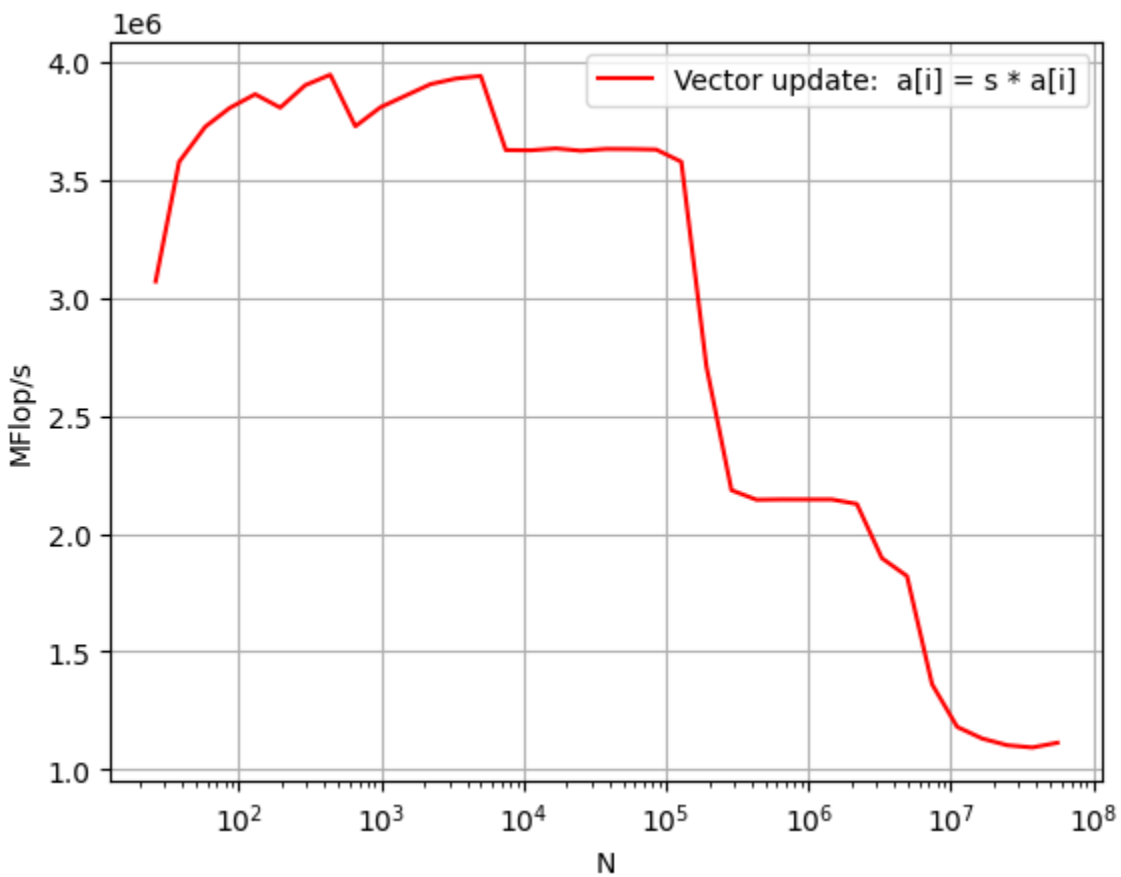
```

MFlop/s	N
3069684.440312	26
3576165.631696	38
3725093.600349	58
3804537.794348	86
3862891.524716	130
3806050.635294	195

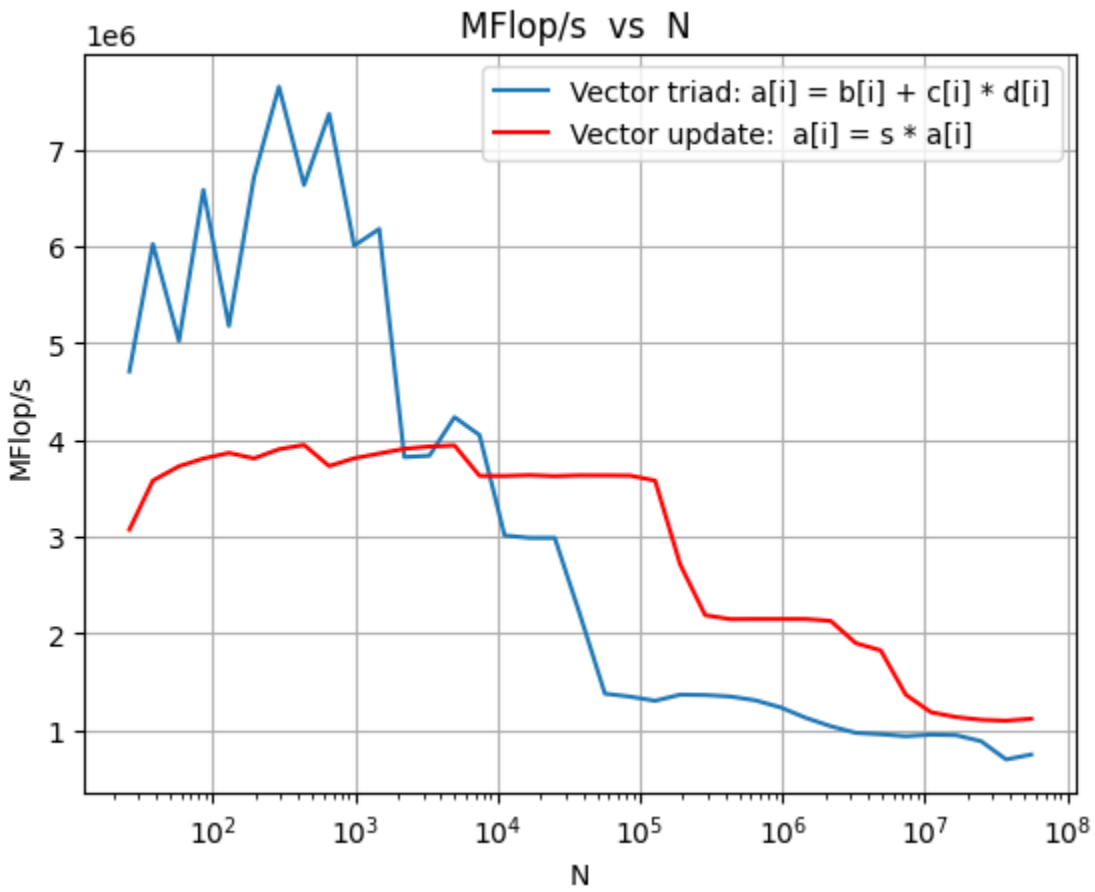
·
·
·

The full list of the output is contained in [./data/v_update.txt](#)

Below shows the graph of th output:



Finally, we combined the 2 graphs together to have a better view of the memory bandwidth with respect to the 2 operations. The graph shown below is our observation.



3) To compile the program with the alternative compiler options, we can use the following commands:

a) Scalar code, NO SIMD:

```
icpx -O3 -no-vec -o double3 Double.cpp timing.c  
Approximate value of Pi: 3.14159  
Time taken: 2.21691 seconds  
CPU cycles per iteration: 4.43382
```

AVX2:

```
icpx -O3 -xHost -o double3 Double.cpp timing.c  
Approximate value of Pi: 3.14159  
Time taken: 1.00476 seconds  
CPU cycles per iteration: 2.00952
```

b) AVX-512:

```
icpx -O3 -xHost -qopt-zmm-usage=high -o double3 Double.cpp timing.c  
Approximate value of Pi: 3.14159  
Time taken: 1.00709 seconds  
CPU cycles per iteration: 2.01419
```

From the results, we can see that using AVX-512 instructions significantly reduces the number of cycles per iteration compared to using scalar code. We can see that the number of cycles per iteration has decreased to 2.015 cycles per iteration from 4.4375. This is because the code is now using more efficient AVX-512 instructions that can perform 8 double-precision divides in a single instruction.

INVERSE THROUGHPUT

Scalar inverse throughput:

CPU cycles per iteration = 4.43382cycles

Number of divide instructions per iteration = 1

Scalar inverse throughput = CPU cycles per iteration / number of divide instructions per iteration

Scalar inverse throughput = 4.43382 cycles/instruction

AVX2 inverse throughput:

CPU cycles per iteration = 2.00952 cycles

Number of divide instructions per iteration = 4

AVX2 inverse throughput = 0.50238 cycles/instruction

AVX-512 inverse throughput:

CPU cycles per iteration = 2.01419 cycles

Number of divide instructions per iteration = 8

AVX-512 inverse throughput = 0.25177 cycles/instruction

Comparing the results, we can see that the AVX-512 double-precision divide instruction has the lowest inverse throughput of 0.25177 cycles/instruction, which is lower than the AVX2 inverse throughput of 0.50238 cycles/instruction and the scalar inverse throughput of 3.58207 cycles/instruction.

A lower inverse throughput means that the instruction is executed faster, resulting in better performance. AVX2 and AVX-512 are both extensions to the basic x86 instruction set, which allow for the use of vectorized instructions. Vectorization is a technique where multiple data elements are processed simultaneously, in parallel.

Overall, a lower inverse throughput is better, as it indicates that the CPU can complete operations more quickly. The vectorized instructions used in AVX2 and AVX-512 can greatly improve performance in applications that can take advantage of them.